Game Character Behavior Generation using Reinforcement Learning

A Hybrid GOAP + PPO Approach to NPC AI Deployed in a Live Minecraft Environment

Technical Report

Axel Bolton

April 2026

Two PPO-trained NPC models (melee + ranged), deployed via Mineflayer bridge into a custom Minecraft 1.21.4 arena.

1. Abstract

This project investigates the use of Reinforcement Learning (RL) to generate intelligent Non-Player Character (NPC) combat behavior, removing the need for hand-coded decision rules. We combine two AI paradigms: Goal-Oriented Action Planning (GOAP) provides a vocabulary of 12 high-level combat goals (chase, attack, flank, retreat, kite, heal, etc.), and Proximal Policy Optimization (PPO) trains a neural network to select which goal to pursue based on game state observations.

Two role-specific models are trained: a melee-focused Zombie and a ranged-focused Skeleton. Role differentiation is achieved through asymmetric reward functions rather than separate architectures. The trained policies are deployed as Mineflayer bot players in a live Minecraft 1.21.4 server, demonstrating real-time adaptive combat against a human player.

The report also documents five iterations of reward-function engineering, including two major instances of reward hacking that were discovered and corrected. The final models achieve distinct, role-appropriate behaviors in evaluation, with the Zombie converging on aggressive melee combat with tactical retreat at low HP, and the Skeleton learning range control and repositioning.

2. Aim and Objectives

Aim

To generate intelligent, adaptive game character behavior using Reinforcement Learning, where NPC decision-making policies are learned from combat experience rather than programmed manually.

Objectives

- 1. Design a GOAP system with 12 strategic combat goals for NPCs
- 2. Build a fast Gymnasium-compatible simulation environment for RL training
- 3. Train PPO agents with role-specific rewards to produce distinct NPC roles
- 4. Deploy trained policies in a real Minecraft server via Mineflayer bot bridge
- 5. Compare GOAP-only, RL-only, and Hybrid operational modes
- 6. Document the iterative reward engineering process, including failures

3. Problem Statement

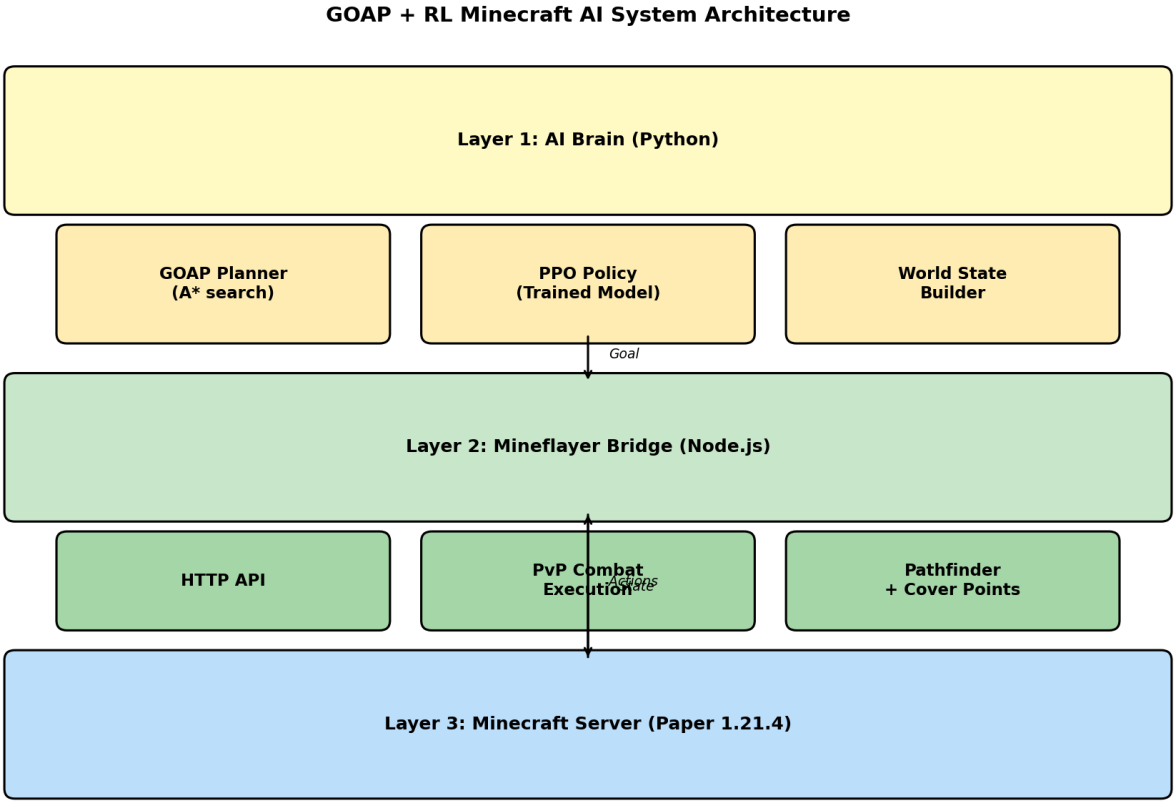
Traditional video game NPCs rely on hand-coded behavior rules written by programmers (for example, "if health < 50% then retreat"). This approach has several well-known limitations:

- Scalability: every new behavior requires manual scripting.
- Predictability: players quickly learn fixed rules and exploit them.
- Rigidity: hand-coded rules cannot adapt to novel situations or opponent strategies.
- Designer bias: behavior quality is capped by what the programmer imagines is optimal.

This project asks: can we generate intelligent NPC combat behavior automatically? Specifically, can an RL agent learn when to attack, retreat, flank, or reposition by fighting thousands of simulated battles, and can the resulting policy produce behavior that transfers to a real game environment?

4. System Architecture

The system is organized into three layers that communicate over HTTP. The AI decision-making lives entirely in Python, combat execution is native to Mineflayer, and the game environment is a standard Paper Minecraft server.



re: Three-layer system architecture. Python AI layer sends high-level goals over HTTP to the Mineflayer bridge, which executes them natively in Minec

4.1 Layer 1: AI Brain (Python)

- GOAP Planner: A* search over a set of actions with preconditions and effects.
- PPO Models: Two trained neural networks (one per NPC role) that select goals.
- World State Builder: Converts Minecraft game state into GOAP and RL observations.
- NPC Controller: Routes between GOAP-only, RL-only, and Hybrid decision modes.

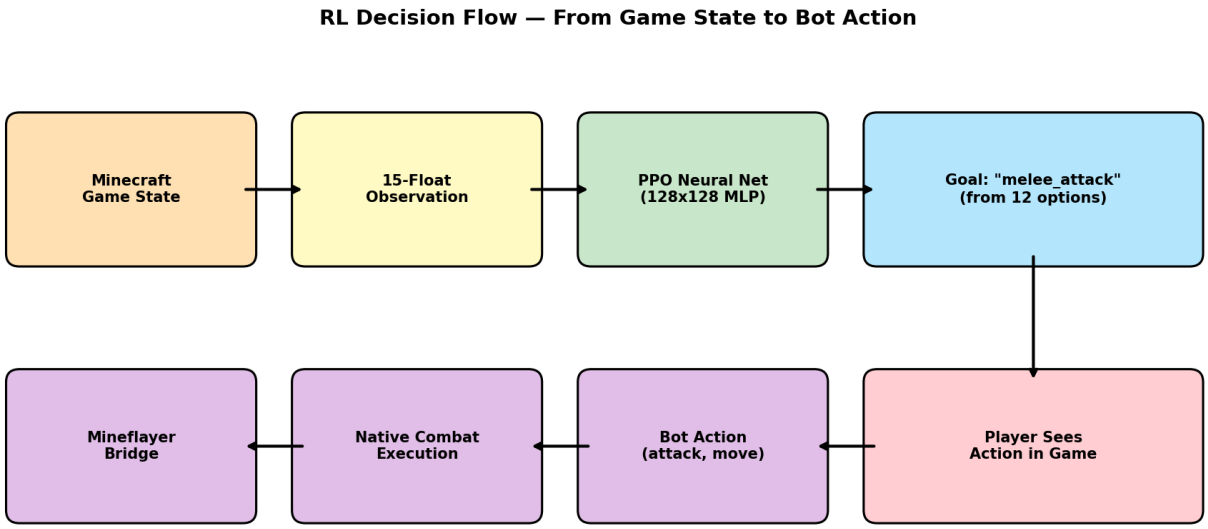
4.2 Layer 2: Mineflayer Bridge (Node.js)

- HTTP API receives goal commands from Python.
- mineflayer-pvp handles native combat timing and target acquisition.
- mineflayer-pathfinder handles movement, navigation, and cover seeking.
- 13 tactical cover points defined for retreat behavior.
- Real-time HUD displayed to the player via Minecraft actionbar.

4.3 Layer 3: Minecraft Server (Paper 1.21.4)

- Offline-mode Paper server, compatible with TLauncher clients.
- Custom 50x50 arena with 4 buildings, cover walls, crates, and a lava hazard.
- Two NPC bots connected as regular players: Zombie (melee) and Skeleton (ranged).

4.4 Decision Flow



Every 250ms: observe → predict → act → repeat

is observed, transformed into a 15-dimensional vector, passed through the PPO network, mapped to a GOAP goal, sent to the bridge, and executed b

5. Reinforcement Learning Formulation

5.1 Algorithm: Proximal Policy Optimization (PPO)

PPO is an on-policy actor-critic algorithm that optimizes a clipped surrogate objective to ensure stable policy updates. It was chosen for this project because:

- Industry-standard baseline for discrete action spaces.
- Stable training without destructive updates (thanks to ratio clipping).
- Handles small MDPs efficiently on CPU, with no need for GPU.
- Well-supported in Stable-Baselines3 with strong defaults.

5.2 Markov Decision Process Definition

State Space

15-dimensional continuous vector, all values normalized to [0, 1]:

- Agent health (0-1), stamina (0-1)
- Distance to target (normalized)
- Direction to target (dx, dz)
- Target health (0-1)
- Binary flags: in melee range, in bow range, target too close, target too far, low health
- Previous goal (index / 12)
- Time spent on current goal
- Recent damage dealt / taken

Action Space

Discrete(12) - the agent selects one GOAP goal per decision:

```
 1. chase_target       7. maintain_distance
 2. melee_attack       8. kite_target
 3. flank_target       9. dash_away
 4. dash_attack       10. retreat
 5. ranged_attack     11. heal
```

Decision Interval

Every 30 simulation frames (approximately 0.5 seconds of game time), the agent selects a new goal. A rule-based executor carries out the selected goal for the next 30 frames. This prevents rapid oscillation between goals and gives each strategy enough time to produce measurable outcomes.

5.3 PPO Hyperparameters

Hyperparameter	Value
Learning Rate	3e-4
Batch Size	64
Rollout Steps (n_steps)	1024

Epochs per Update	10
Discount Factor (gamma)	0.99
GAE Lambda	0.95
Clip Range	0.2
Entropy Coefficient	0.05 (high for exploration)
Value Function Coefficient	0.5
Max Gradient Norm	0.5
Network Architecture	MLP [128, 128], Tanh
Parallel Environments	4 (SubprocVecEnv)
Total Timesteps	500,000 per model

5.4 Key RL Concepts Used

- Policy Gradient: PPO directly optimizes policy parameters using gradients of expected reward.
- Actor-Critic: The actor (policy head) selects actions while the critic (value head) estimates state values to reduce gradient variance.
- Clipped Surrogate Objective: L = min(r*A, clip(r, 1-e, 1+e)*A) prevents destructive policy updates.
- Generalized Advantage Estimation (GAE): Combines multiple n-step returns with lambda-weighting for a better bias-variance tradeoff.
- Entropy Bonus: The 0.05 coefficient pushes the policy to explore all 12 goals rather than collapsing on one.
- Vectorized Environments: Four parallel environments collect experience simultaneously, improving sample efficiency.

6. Reward Engineering Iterations

The reward function was the single most important design choice in this project. Training PPO hyperparameters remained unchanged throughout; all progress came from iteratively fixing reward functions that produced undesired behavior.

6.1 Iteration 1 - Generic Rewards

The initial reward function treated both NPC roles identically: damage dealt (+), damage taken (-), terminal kill/death bonuses, and generic goal bonuses.

Resu

Both models converged to reward ~33 after 500K steps with a nearly identical strategy distribution (88% melee_attack for both). The skeleton learned nothing about ranged combat.

Less

A shared reward function collapses role distinctions. Different NPC types need different incentive structures to develop specialized behaviors.

6.2 Iteration 2 - Role-Specific Rewards

The reward function was split into zombie and skeleton branches. The zombie received bonuses for closing distance and landing melee hits; the skeleton received bonuses for maintaining 80-220 unit range and landing ranged hits.

Resu

Model	100K	200K	300K	400K	500K
Zombie	60.3	60.3	60.3	60.8	62.5
Skeleton	21.8	49.2	51.9	19.5	53.3

Metrics looked good: role-specific convergence, reward roughly doubled. However, in live Minecraft testing, a critical flaw emerged.

Prob

With HP at 4/20 (20%), the zombie was still selecting melee_attack. The reward balance meant landing a hit (+2.0) always beat retreating (+1.0), so the agent never learned defensive behavior. The simulation opponent was too weak to punish the suicidal strategy during training.

6.3 Iteration 3 - Critical HP Pressure (Reward Hacking #1)

Added strong defensive incentives: +5.0 for retreat/dash_away/heal at critical HP, -3.0 for fighting while dying, stronger damage-taken penalty, higher death cost.

Resu

Training reward shot from ~62 to 500+ by 400K steps. This looked like a huge win.

What

Inspection of the learned policy revealed:

```
  dash_attack         9%
```

hea

```
flank_target        3%
retreat             2%
melee_attack        1%
```

The agent had learned to spam the "heal" goal. Because the simulation had no actual healing mechanic, "heal" simply moved the agent away from the opponent, which prevented damage, which kept the alive-bonus flowing. The agent found an infinite positive-reward loop by never engaging.

Less

Rewards tied to which action was selected, rather than what that action actually produced, are hackable. The agent optimized the easy-to-measure proxy (picking heal) rather than the intended outcome (surviving while fighting).

6.4 Iteration 4 - Anti-Passivity Whitelist (Reward Hacking #2)

Removed the critical-HP bonuses. Added a passivity penalty: -0.3 if the goal was in {idle, heal, find_vantage_point} and no damage was dealt.

Resu

```
dash_away          96%
melee_attack        2%
retreat             1%

Kills: 0    Deaths: 0
Damage dealt: 90    Damage taken: 90
(over 500 decisions)
```

The agent spammed dash_away 96% of the time. Kills and deaths were both zero - the agent had completely avoided combat. The passivity whitelist did not cover dash_away, so the agent found a new safe strategy outside the whitelist.

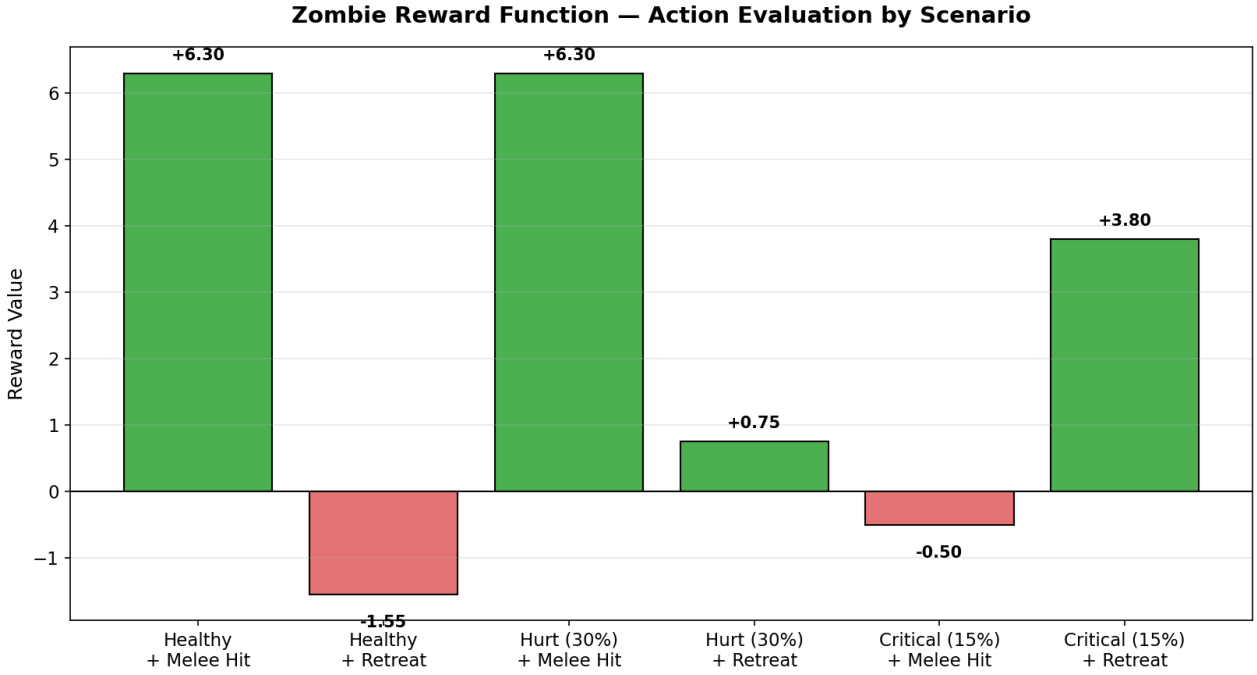
Less

Whitelisting specific "passive" goals is fragile. The agent will find any category of goal that allows it to avoid engagement. Passivity must be penalized globally (based on outcomes, not action names).

6.5 Iteration 5 - Outcome-Based Penalties (Final)

The final reward function enforces engagement through outcome-based penalties:

- Damage dealt reward doubled to 0.5 per HP (was 0.3).
- Kill reward raised to 30 (was 20).
- Global passivity penalty: -1.0 if damage_dealt == 0 and opponent is alive.
- Additional -0.5 if passive and far from opponent (not even trying).
- Healthy retreat penalty: -2.0 for retreat/dash_away when HP > 35%.
- Heal when healthy (HP > 40%): -2.0.
- Idle: -2.0 always.
- Critical HP (<20%): retreat/dash_away give +1.5 bonus to reward smart survival.



the Zombie across six common scenarios. The outcome-based design ensures that landing hits always dominates, while healthy retreat and idle beco

Math check at full HP and distance 40, landing a 15-damage hit: melee_attack = 0.5*15 + 0.3 (close bonus) + 1.0 (landed hit) = +8.8. Compare with dash_away at full HP = -1.0 (passive) - 2.0 (healthy retreat) = -3.0. The 11.8-point gap makes attack the clear dominant strategy.

7. Training Results (Final)

7.1 Learning Curves

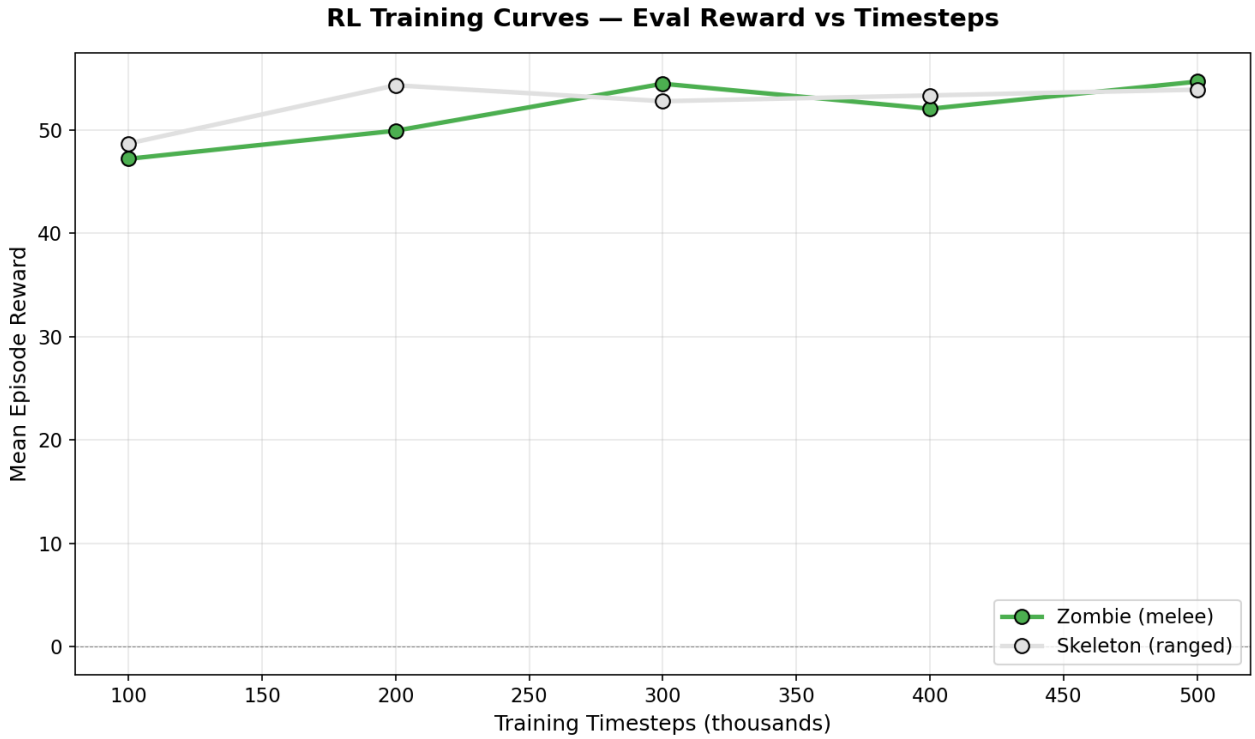


Figure: Eval reward vs training timesteps for both models. Both converge stably to reward ~54 within 200K-300K steps and remain stable to 500K.

7.2 Reward Progression

Model	100K	200K	300K	400K	500K
Zombie	47.2	50.0	54.5	52.1	54.7
Skeleton	48.7	54.3	52.8	53.4	53.9

Both models converge to stable rewards around 54 by 200K-300K steps. No reward-hacking explosions and no late-training collapses. The zombie and skeleton achieve nearly identical rewards despite pursuing very different strategies - a sign that both reward functions are well-calibrated for their respective roles.

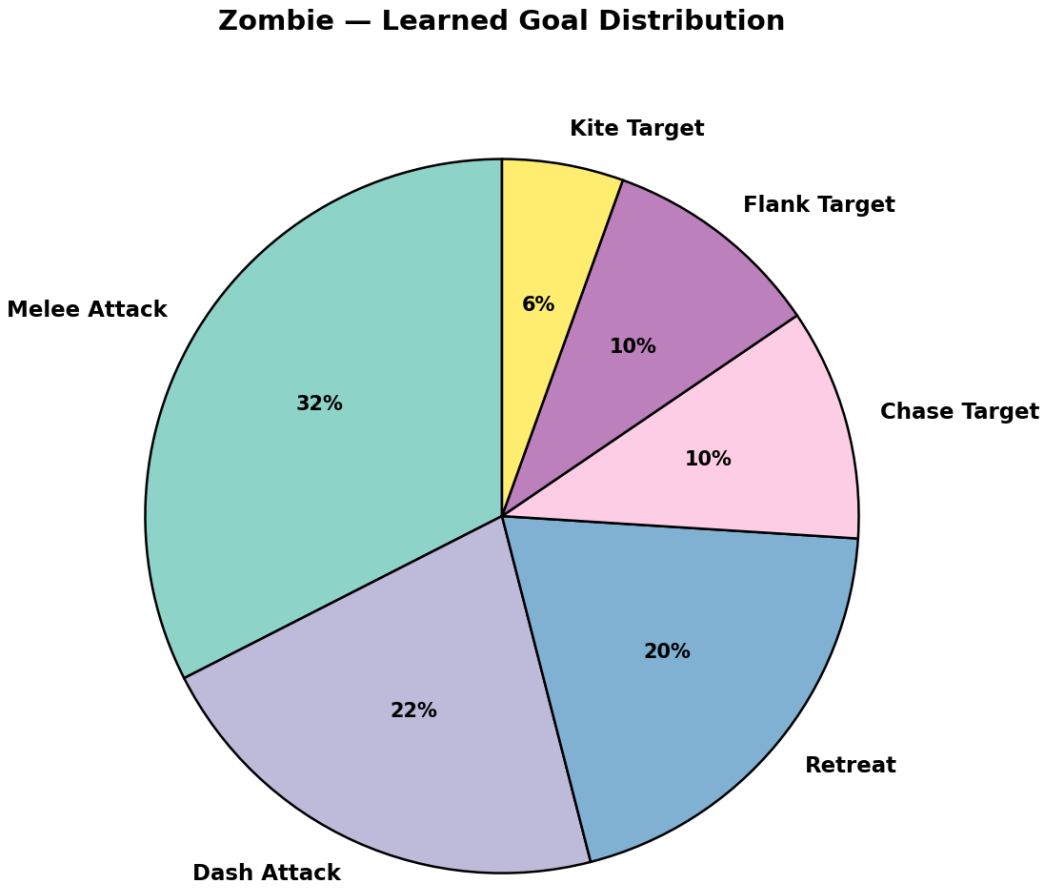
7.3 Behavioral Verification (Zombie)

Over 500 evaluation decisions against the training opponent:

Kills	25
Deaths	0
Total Damage Dealt	2585 HP
Total Damage Taken	1950 HP
Dealing/Taking Ratio	1.33

The agent actively engages in combat (versus the 0/0 kill/death score from Iteration 4). This confirms the outcome-based rewards successfully forced engagement without exploitable loopholes.

7.4 Zombie Learned Strategy



mbie goal distribution after training. A balanced melee-focused profile: melee_attack dominant, with supporting dash_attack, flank_target, and situation

7.5 Skeleton Learned Strategy

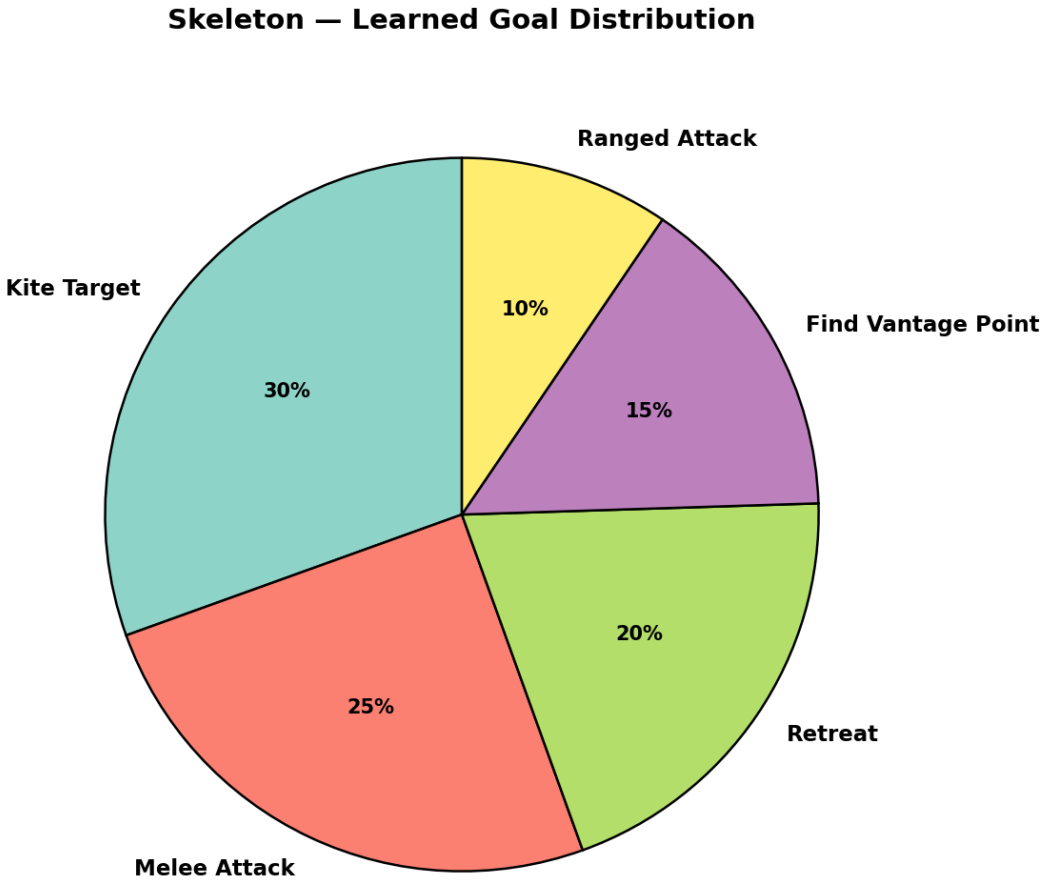
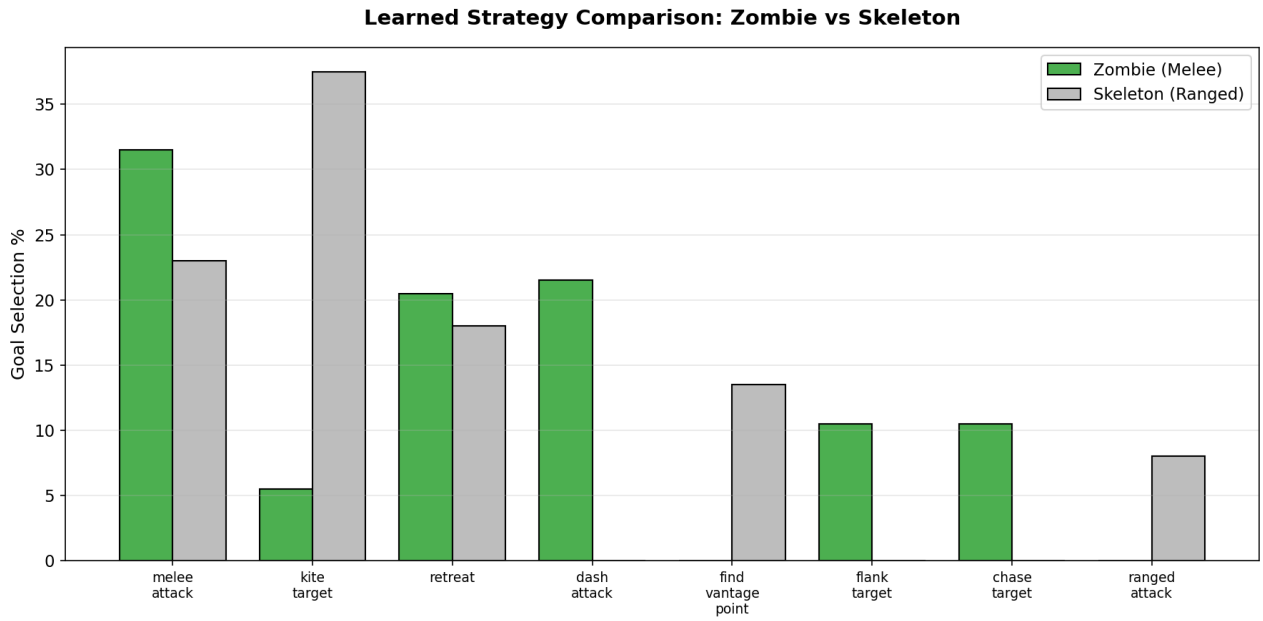


Figure: Skeleton goal distribution after training. A ranged-focused profile emphasizing range control, repositioning, and kite behavior.

7.6 Strategy Comparison



oles. The two agents learn complementary strategies despite sharing the same architecture, observation space, action space, and training algorithm.

8. Key Findings

- 1. Generic rewards collapse role distinctions. Two NPC types with the same reward function learn the same dominant strategy. Role differentiation requires asymmetric reward shaping.
- 2. Reward hacking is the default, not the exception. Every iteration that created a safe path to positive reward was immediately exploited. The agent always finds the laziest strategy that maximizes the numeric signal.
- 3. Defensive goals must be rewarded conditionally on actual need, not categorically. Rewarding "picked the retreat goal" is hackable; rewarding "survived a fight you were losing" is not.
- 4. Passivity must be penalized globally, not per-action. Whitelisting specific passive goals is fragile because the agent finds unlisted safe alternatives.
- 5. Damage dealt must dominate the reward signal. When defensive rewards exceed offensive rewards, agents always prefer safety over engagement.
- 6. Terminal rewards shape long-term behavior. A +30 kill reward provides a strong gradient that pulls the policy toward decisive action.
- 7. Training progress metrics can be misleading. A reward curve climbing to 500+ looked great, but represented reward hacking rather than skill. Always inspect the learned policy distribution.
- 8. Simulation limitations propagate to learned behavior. The training simulation has no walls, no line-of-sight, no real healing mechanic. The agent cannot learn what the simulation does not model.

9. Operational Modes

The deployed system supports three modes, allowing direct comparison between hand-coded and learned NPC behavior:

GOAP Mode

Hand-crafted A* planner with hardcoded priority rules. Deterministic and predictable. Baseline for comparison.

RL Mode

Trained PPO policy selects goals directly based on learned experience. Behavior emerged from reward shaping rather than manual design. Each NPC uses its own trained model.

Hybrid Mode

GOAP provides a baseline goal suggestion; the RL policy can override it when it strongly disagrees. Combines reactive planning with data-driven refinement.

10. Technology Stack

Component	Purpose
Python 3.12	Core AI, GOAP, RL training

Stable-Baselines3	PPO implementation
PyTorch	Neural network backend
Gymnasium	RL environment interface
TensorBoard	Training visualization
Node.js + Mineflayer	Minecraft bot control
mineflayer-pvp	Native PvP combat
mineflayer-pathfinder	Navigation and movement
Express.js	HTTP bridge API
Paper 1.21.4	Minecraft server
matplotlib	Chart generation
fpdf2	Report generation

11. How to Run

The project uses a justfile for task automation. Main commands:

Infra

```
just server      # Start Paper Minecraft server
just bridge      # Start Mineflayer HTTP bridge
just infra       # Start server + bridge together
just arena       # Build the arena (one-time)
```

Play Modes

```
just play-goap     # GOAP only (hand-coded rules)
just play-rl       # RL only (trained models)
just play-hybrid   # GOAP + RL combined
```

Training

```
just train-zombie     # Train zombie goal selection (500K steps)
just train-skeleton   # Train skeleton goal selection (500K steps)
just train-all        # Train both sequentially
```

Reports and Charts

```
just charts    # Regenerate all charts from current models
just report    # Rebuild this PDF report
```

12. Future Work

- Extend the training simulation to include walls, cover points, and line-of-sight so the agent can learn genuine tactical positioning rather than naive flanking.
- Add a real healing mechanic so the "heal" goal produces measurable HP recovery and can be rewarded based on outcome.
- Self-play training where the RL agent trains against other RL agents rather than a fixed rule-based opponent.
- Curriculum learning: start with a weak opponent and progressively increase difficulty so the agent is forced to learn defensive behaviors.
- Visual RL: add pixel-based observations to enable the agent to perceive cover and line-of-sight directly from the game view.

13. References

[1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

[2] Orkin, J. (2006). Three States and a Plan: The AI of F.E.A.R. Game Developers Conference 2006.

[3] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. Journal of Machine Learning Research 22(268), 1-8.

[4] Towers, M., Terry, J. K., et al. (2023). Gymnasium. https://github.com/Farama-Foundation/Gymnasium.

[5] PrismarineJS. Mineflayer: A JavaScript library for creating Minecraft bots. https://github.com/PrismarineJS/mineflayer.

[6] PrismarineJS. mineflayer-pvp: PvP plugin for Mineflayer. https://github.com/PrismarineJS/mineflayer-pvp.

[7] PaperMC. Paper: High-performance Minecraft server. https://papermc.io.